Render Watcher Agent

A Flutter mobile app with an AI agent that watches your Render production logs, tells you when something looks wrong, and asks what to do.

1. The idea in one paragraph

Developers deploy to Render but rarely watch logs all day. This app connects to a user's Render services, reads their production logs in the background, and uses an LLM agent to judge which log lines are suspicious or serious. When the agent finds something, it sends a push notification (via FCM) and asks the user what to do: ignore it, view the logs, or run an approved action such as a restart. The agent only acts inside limits the user sets, and when it is unsure it asks instead of acting.

2. How it works

Step	What happens
Sign in	User signs in with GitHub (identity only).
Connect	User taps +, pastes a Render API key once. Backend stores it encrypted and lists their services.
Pick services	User chooses which services to watch. Logs appear live in the app.
Watch	Backend reads logs through the Render Logs API (query or websocket subscribe) and filters for errors, warnings, 5xx spikes, crash loops and memory problems. Repeated lines are grouped into one alert.
Triage	LLM agent rates severity and suggests a likely cause.
Confidence gate	If sure, it alerts directly. If unsure, it says so and asks the user to check.
Notify	Backend sends a push via FCM (Firebase Admin SDK). Flutter app uses firebase_messaging. iOS also needs an APNs key.
Respond	Notification buttons: Ignore, Show logs, Restart service. Taps call the backend, which uses the Render API inside the user's limits.

3. What makes it different

- **Mandate-bound actions:** the user signs off on which actions the agent may take (for example, restart only). Anything else needs a tap.
- **Confidence-gated autonomy:** the agent acts or alerts on its own only when it is confident; otherwise it asks. Statistical methods such as conformal prediction could back this gate (research direction, not yet validated).
- **Optional verifier agent:** a second pass that tries to refute the first agent's diagnosis before the user is alerted.
- **Audit log:** every alert and action is recorded with the evidence behind it.

4. Render integration notes

- Render authenticates API requests with an API key from Account Settings. I did not find a 'Login with Render' OAuth option.

- GitHub cannot be the link to Render: services are not discoverable through GitHub, so GitHub is for sign-in only.
- Render has a Webhooks API (create webhook via POST /webhooks), but webhooks require a Professional plan or higher, and a Professional workspace can push to only one destination URL. They cover service events (deploy started, scaling down), not log lines.
- Plan: make webhooks optional (deploy-failure alerts). Build the core on the Logs API, which does not need them.

5. Security and risks

- A Render API key gives access to all workspaces the user belongs to. Store keys encrypted on the backend only, never in the app. Tell users they can revoke the key at any time.
- Rate-limit and group alerts, or users will ignore them.
- Run the watcher outside Render so it does not go down with the service it watches.
- Version 1 should only notify. Do not let the agent restart anything yet.
- Market gap and competitors (existing log and alerting tools) have not been researched yet.

6. Suggested stack

Flutter app (firebase_messaging) | FastAPI backend | Postgres | Render Logs API | LLM for triage | Firebase Cloud Messaging for push.

7. MVP scope

- Connect your own Render account only.
- Watch 1-3 services, show live logs in the app.
- Rule-based filter plus LLM triage, push notification on serious issues.
- Read-only: Ignore and Show logs buttons. Add Restart later, behind a mandate.

Notes on Render and FCM details come from Render's public docs and general knowledge of Firebase; verify against current docs before building.